

Virtual Memory

中国人民大学信息学院

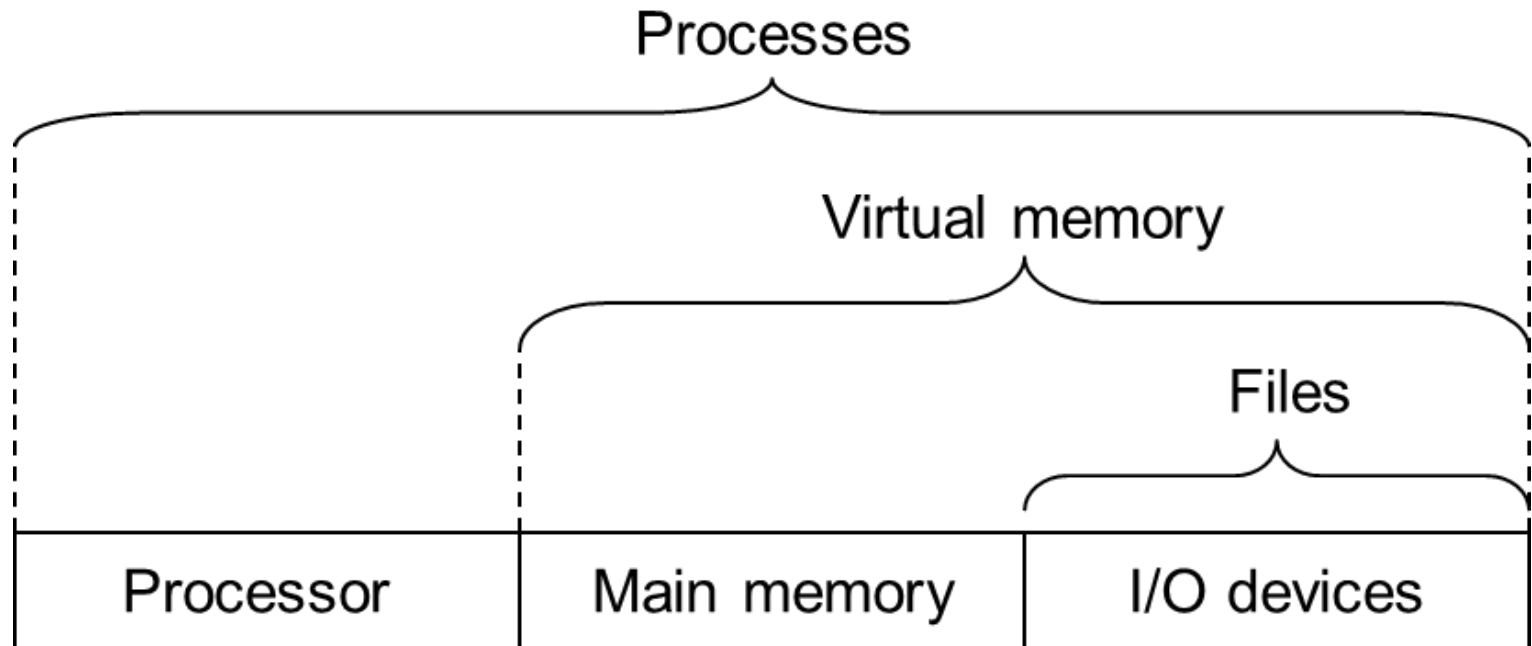
卞昊穹

bianhq@ruc.edu.cn

内部资料，请勿外传

OS: Three Easy Pieces

- 虚拟化
- 持久化
- 并发



Virtualization

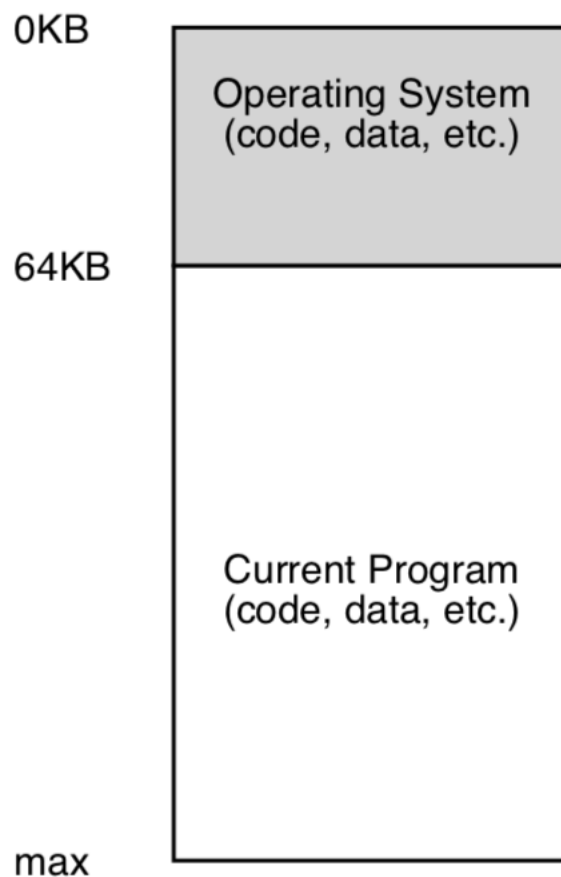
- 计算机主要硬件
 - CPU（运算器/控制器）
 - 进程
 - 内存（存储器）
 - 虚拟内存
 - 外存（输入输出）
 - 文件系统

Outline

- VM History
- VM Overview
- Memory API
- VM Discussions

Early Systems

- 直接使用物理地址，没有抽象
- **OS**还只是一些库，放在内存地址**0**开始的一段地址空间
- 用户程序从某个地址开始（如**64K**），使用剩余的全部内存
- 用户程序都要自己搞定，对**OS**没有多少期待



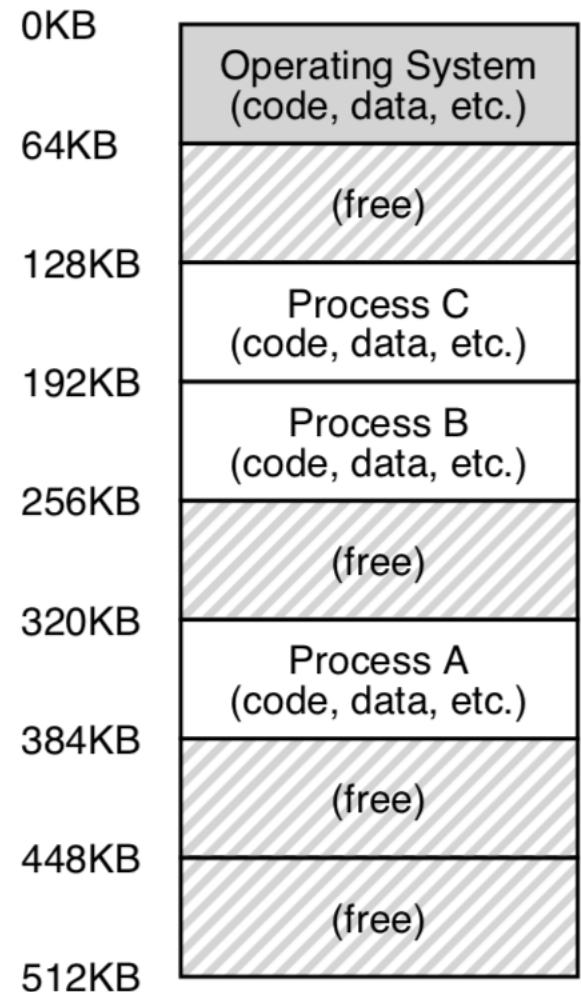
Multiprogramming and Time Sharing

- Multiprogramming
 - 多个程序并发（不并行），当有程序在进行Io操作时，其他进程可以利用CPU，提高CPU利用率
- Time Sharing
 - Batch processing的反应太慢，用户感受不好
 - 例如debug过程反馈时间很长
 - 让用户感觉自己独享整个计算机
 - 用户的interactivity开始重要

Time Sharing Implementation

- **Method 1.** 每次一个程序独占全部内存
 - 好处：内存更大，程序执行更快
 - 缺点：切换任务时，需要有大量的I/O，因此性能很差
- **Method 2.** 每个程序都留在内存中
 - 各占用一部分内存

三个进程共享内存

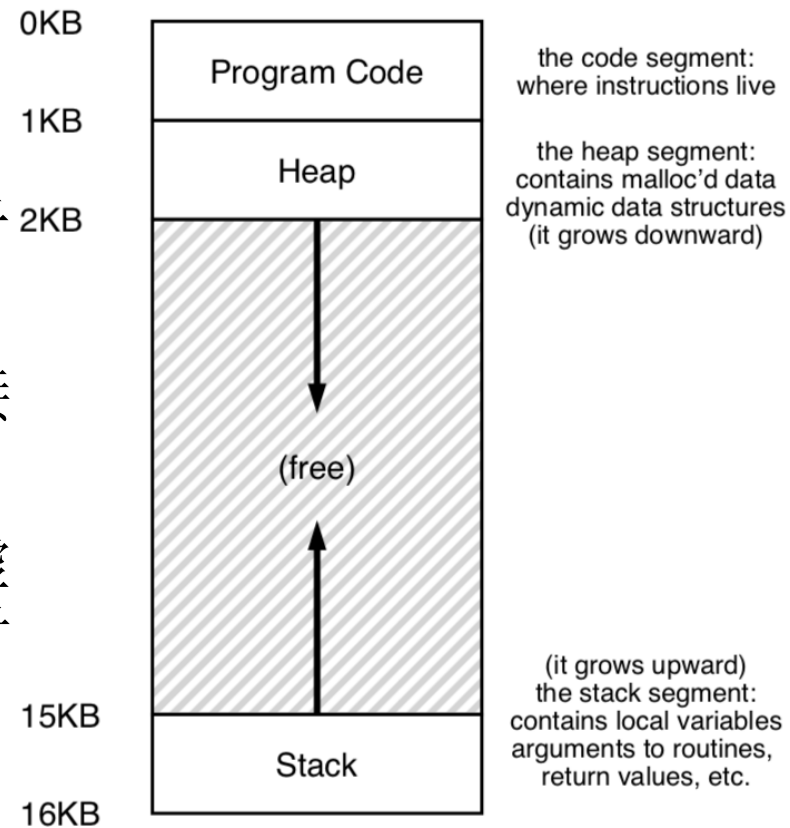


Multi-program -> Protection

- 为了不让进程之间互相访问(huai)对方的内存

- **Abstraction: address space**

- 每个进程应该只能看到属于自己的内存空间
- 那么就不能把物理地址直接给用户
- 建立虚拟地址空间，以及虚拟地址和物理地址间的映射

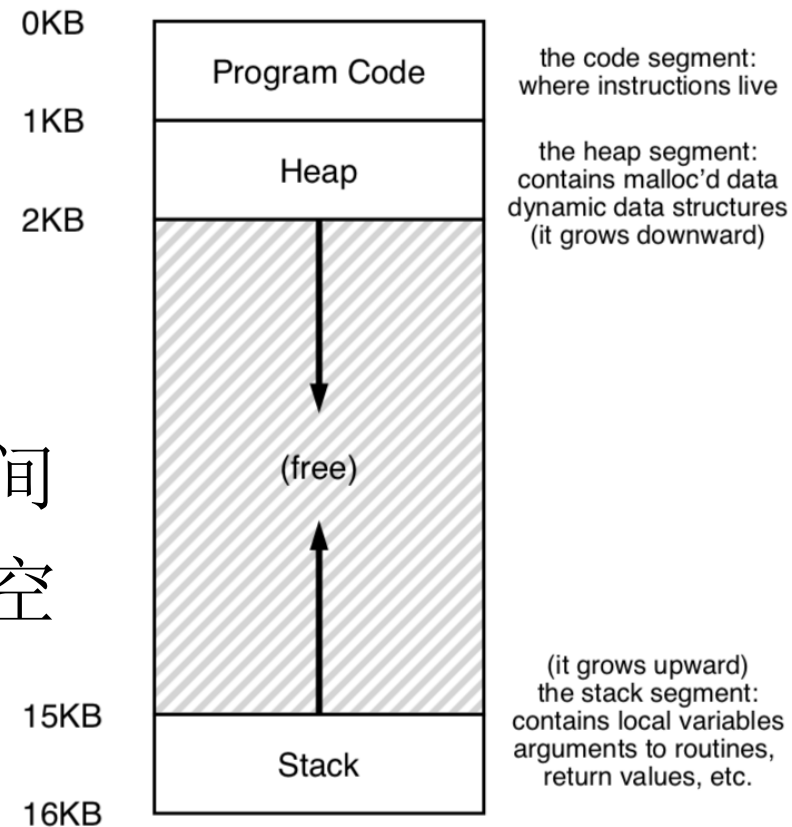


Multi-program -> Protection

- Address space

- Code区域
- Stack区域
- Heap区域

- 看不到其他进程的内存空间
- 也看不到内核程序的内存空间



VM Overview

Virtual Memory

- **Abstraction**
 - 假的，但是方便使用
- **Transparent**
 - 用户程序感知不到，以为是真的
 - 在物理地址和虚拟地址上都能跑
- **Efficient**
 - 否则用户不太能接受VM机制（性能是前提）
- **Protection / Isolation**
 - 进程无法访问其他进程、内核的地址空间
 - 任意一条指令执行时，不能访问到自己空间以外的地方

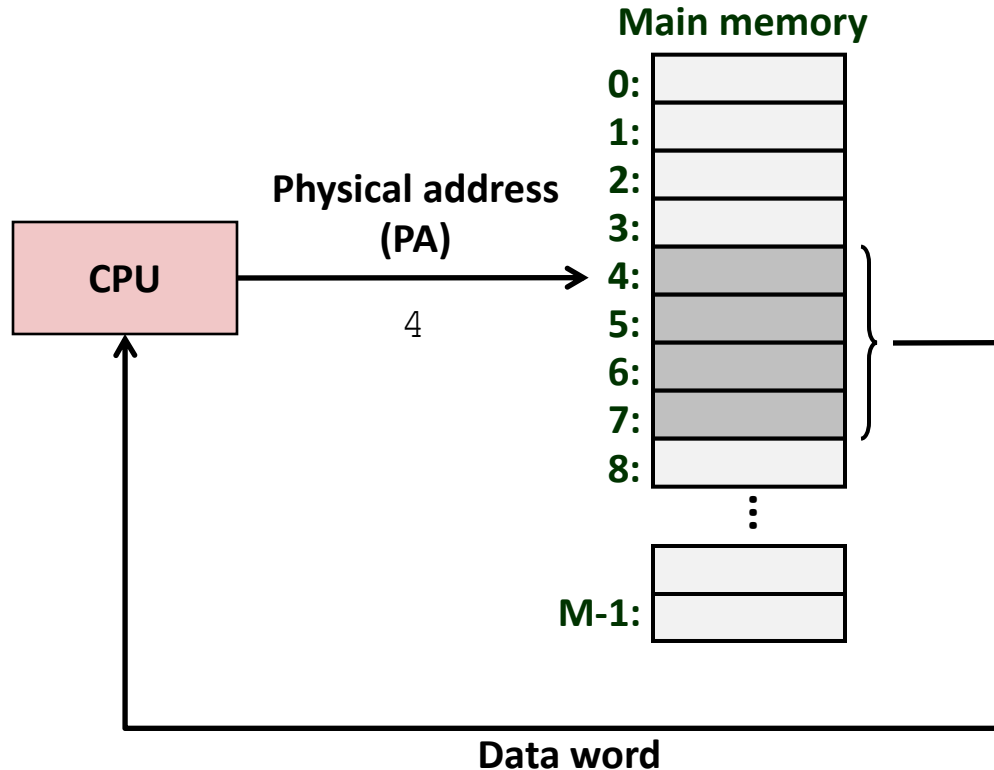
Virtual Memory

- Addresses
 - Virtual address
 - Logical address
 - Physical address

Physical Addressing

- Attributes of the main memory
 - Organized as an array of M contiguous byte-sized cells
 - Each byte has a unique **physical address** (PA) started from 0
- physical addressing
 - A CPU use physical addresses to access memory
- Examples
 - Early PCs, DSP, embedded microcontrollers, and Cray supercomputers

A System Using Physical Addressing



- Used in "simple" systems like embedded microcontrollers in devices like cars, elevators, and digital picture frames

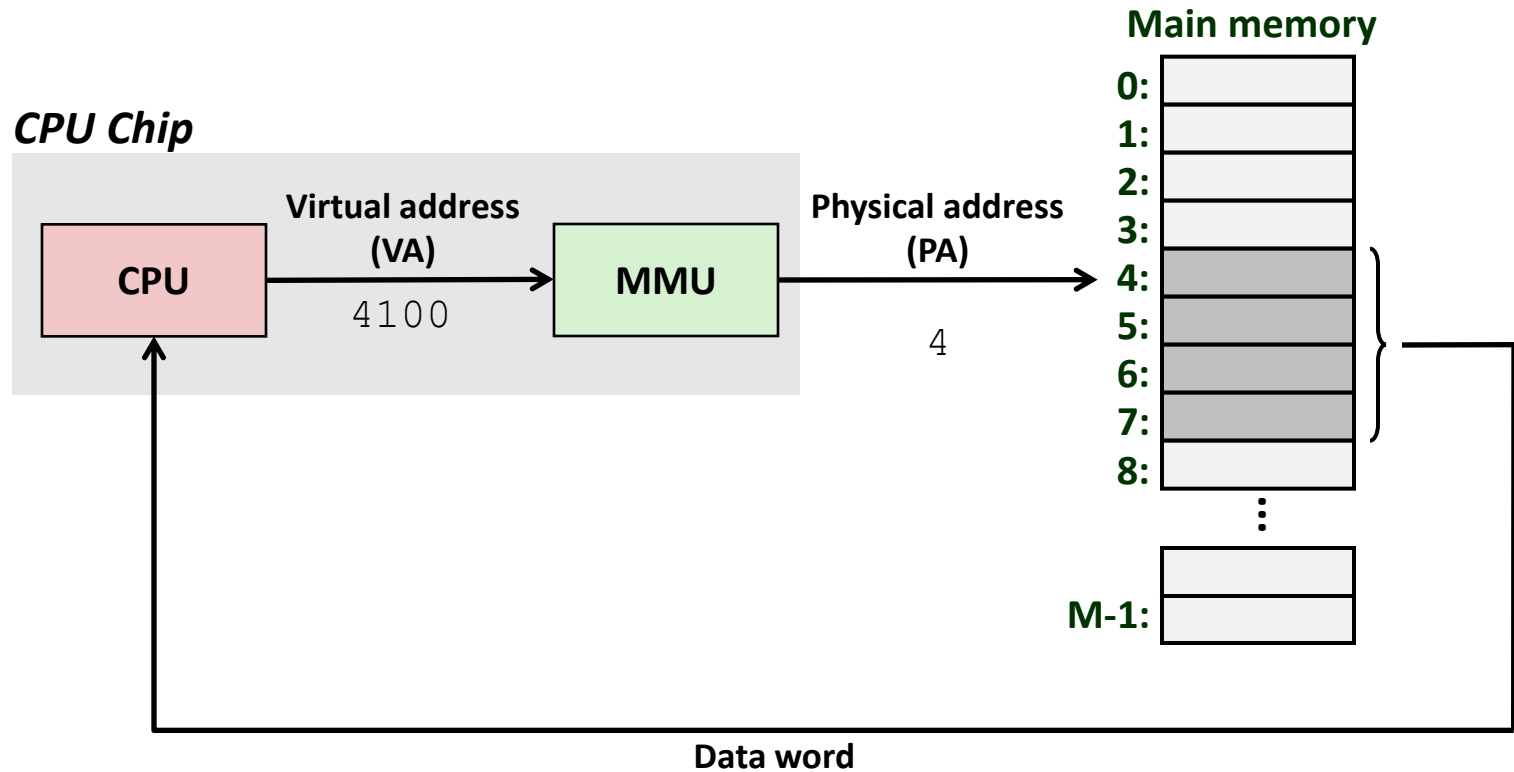
Virtual Addressing

- Virtual addressing
 - the Program accesses main memory by a **virtual address** (VA)
 - The virtual address is converted to the appropriate physical address by hardware

Virtual Addressing

- Address translation
 - Converting a **virtual address** to a **physical address**
 - Requires close **cooperation** between the CPU hardware and the operating system
 - HW: the memory management unit (MMU)
 - Dedicated hardware on the CPU chip to translate virtual addresses on the fly
 - SW: A look-up table (MMU可以访问)
 - Stored in main memory
 - Contents are managed by the operating system

A System Using Virtual Addressing



- Used in all modern servers, desktops, and laptops
- One of the great ideas in computer science

Address Space

- N-bit Address Space
 - Virtual address space:
Set of $N = 2^n$ virtual addresses $\{0, 1, 2, 3, \dots, N-1\}$
 - Physical address space:
Set of $M = 2^m$ physical addresses $\{0, 1, 2, 3, \dots, M-1\}$
- Each object can now have multiple addresses
 - one physical address, one (or more) virtual addresses

Virtual Memory

- 打印程序中各部分地址
 - 打印出来的都是virtual address

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  int main(int argc, char *argv[]) {
4      printf("location of code   : %p\n", (void *) main);
5      printf("location of heap   : %p\n", (void *) malloc(1));
6      int x = 3;
7      printf("location of stack  : %p\n", (void *) &x);
8      return x;
9  }
```

- 64-bit Mac OS:
 - 先是code，然后是heap，再是stack

```
location of code   : 0x1095afe50
location of heap   : 0x1096008c0
location of stack  : 0x7fff691aea64
```

*

练习4-1

- 填写下表中的空格和?
- $K=2^{10}$ (Kilo), $M=2^{20}$ (Mega), $G=2^{30}$ (Giga),
 $T=2^{40}$ (Tera), $P=2^{50}$ (Peta), $E=2^{60}$ (Exa)

#virtual address bits (n)	#virtual address (N)	Largest possible virtual address
8		
	64K	
		$2^{32}-1=?G-1$
	$2^?=256T$	
64		*

练习4-1答案

- $K=2^{10}$ (Kilo), $M=2^{20}$ (Mega), $G=2^{30}$ (Giga),
 $T=2^{40}$ (Tera), $P=2^{50}$ (Peta), $E=2^{60}$ (Exa)

#virtual address bits (n)	#virtual address (N)	Largest possible virtual address
8	256	255
16	64K	64K-1
32	4G	4G-1
48	256T	256T-1
64	16E	16E-1

Memory API

Memory API

- Stack:
 - 申请局部变量

```
void func() {  
    int x; // declares an integer on the stack  
    ...  
}
```

- 全局变量区
 - 全局变量，静态变量

Memory API

- Heap:
 - Malloc(), free()

```
int *x = malloc(10 * sizeof(int));  
...  
free(x);
```

```
#include <stdlib.h>  
...  
void *malloc(size_t size);
```


Memory API

- **Malloc(), free()**不是**system call**
 - 只是**library call**, **C**函数库内部通过一定的结构来保存当前有多少可用内存
 - 如果程序 **malloc**的大小超出了库里所留存的空间, 那么将首先调用**brk**系统调用来增加可用空间, 然后再分配空间
 - 调用其他**system call**: **brk, mmap**
 - **free**时, 释放的内存并不立即返回给**os**, 而是保留在内部结构中
- 系统调用通常提供一种最小功能, 而库函数通常提供比较复杂的功能。

Memory API

- `brk`
 - 改变heap区大小的system call, 修改heap end指针
 - 不要直接调用, 一般都是内存分配的库函数调用
- 此外还用到了mmap系统调用
 - 后面详细介绍

Memory API

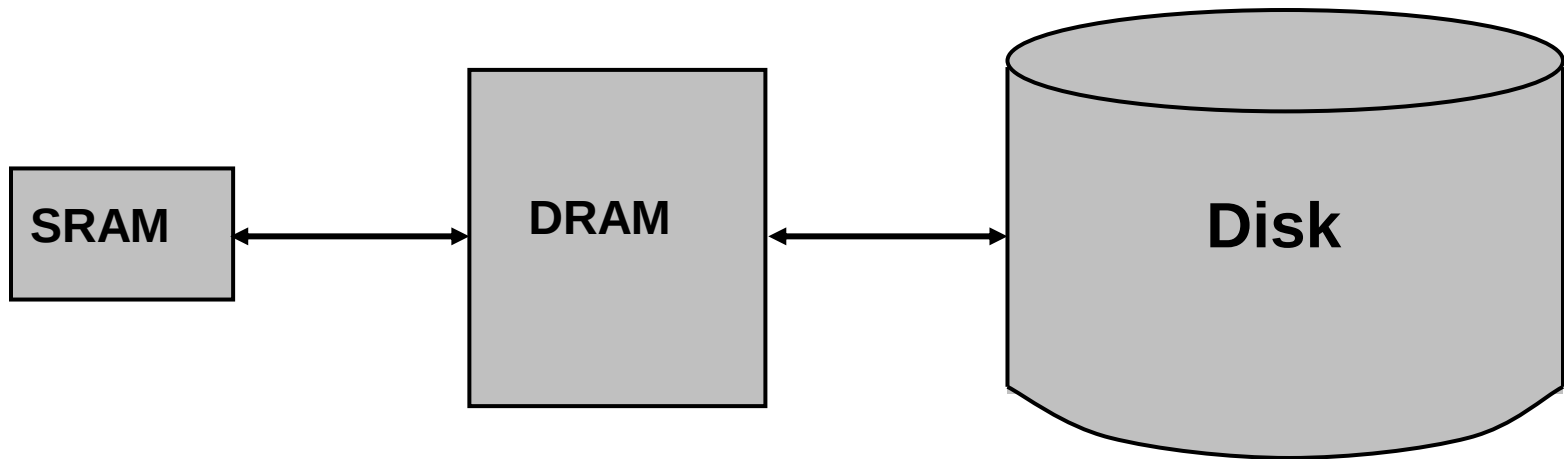
- `Calloc()`
 - `Malloc()` + 初始化为0
- `Realloc()`
 - 重新分配空间，扩充分配空间
 - 具体操作是找到一个更大的空间，然后把旧的数据复制过去，返回新空间的指针

VM Discussions

为什么使用VM?

- 内存高效使用
 - 将**DRAM**作为虚拟地址空间的缓存
- 简化内存管理
 - 每个进程都得到相同的线性地址空间
- 隔离内存空间
 - 进程之间的内存空间互不影响
 - 用户程序无法访问内核中的内存空间

Using Main Memory as a Cache



Using Main Memory as a Cache

- DRAM vs. disk is more extreme than SRAM vs. DRAM
 - Access latencies:
 - DRAM ~10X slower than SRAM
 - Disk ~100,000X slower than DRAM

Design Considerations

- Line size? (Large vs. Small)
 - **Large**, since disk better at transferring large blocks
- Associativity? (Full vs. Direct)
 - **Full**, to minimize miss rate (全相连/组相连/直接映射)
- Write through or write back? (WB vs. WT)
 - **Write back**, since can't afford to perform small writes to disk

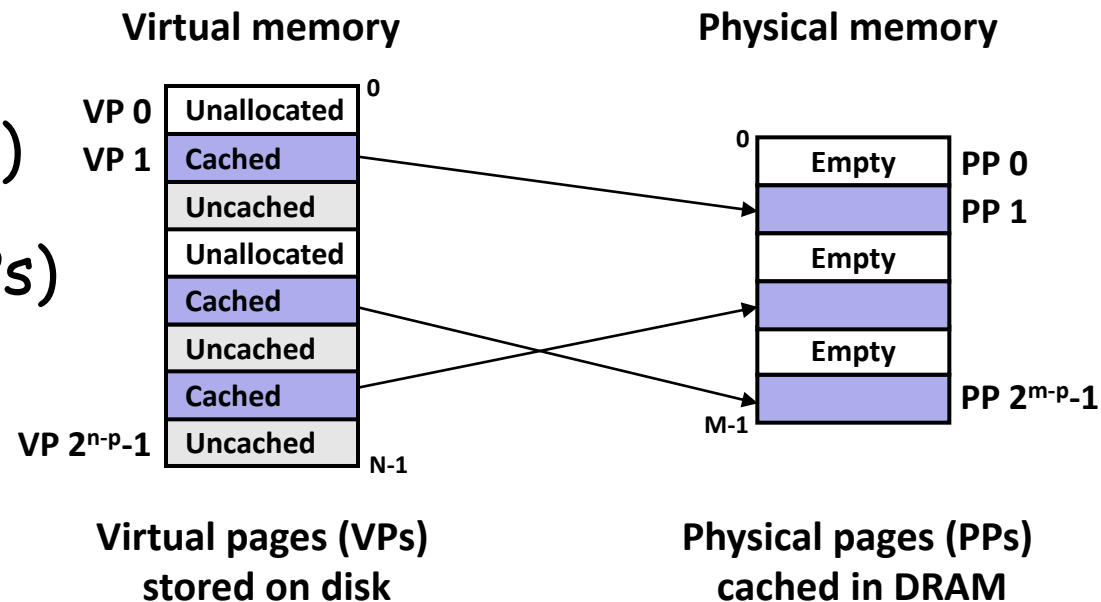
Page

- Virtual memory
 - Organized as an **array** of contiguous byte-sized cells stored on disk **conceptually**
 - Each byte has a unique **virtual address** that serves as an index into the array
 - The contents of the array on disk are **cached** in main memory

Page

- The data on disk is partitioned into **blocks**
 - Serve as the transfer units between the disk and the main memory

- virtual pages (VPs)
- physical pages (PPs)
 - or page frames



Page Attributes

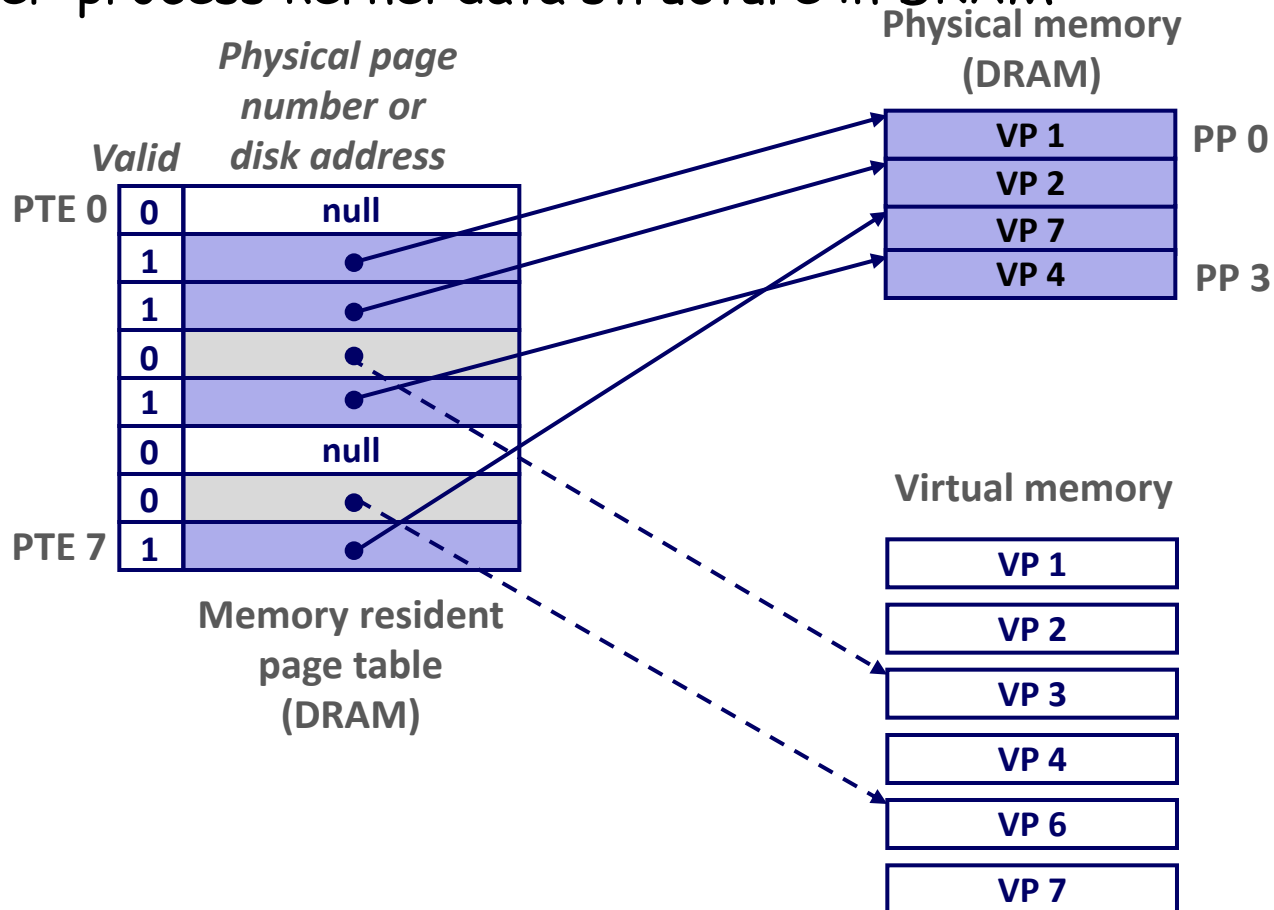
- Unallocated:
 - Pages that have not yet been allocated (or created) by the VM system
 - Do not have any data associated with them
 - Do not occupy any space on disk.

Page Attributes

- **Cached:**
 - Allocated pages that are currently cached in physical memory.
- **Uncached:**
 - Allocated pages that are not cached in physical memory.

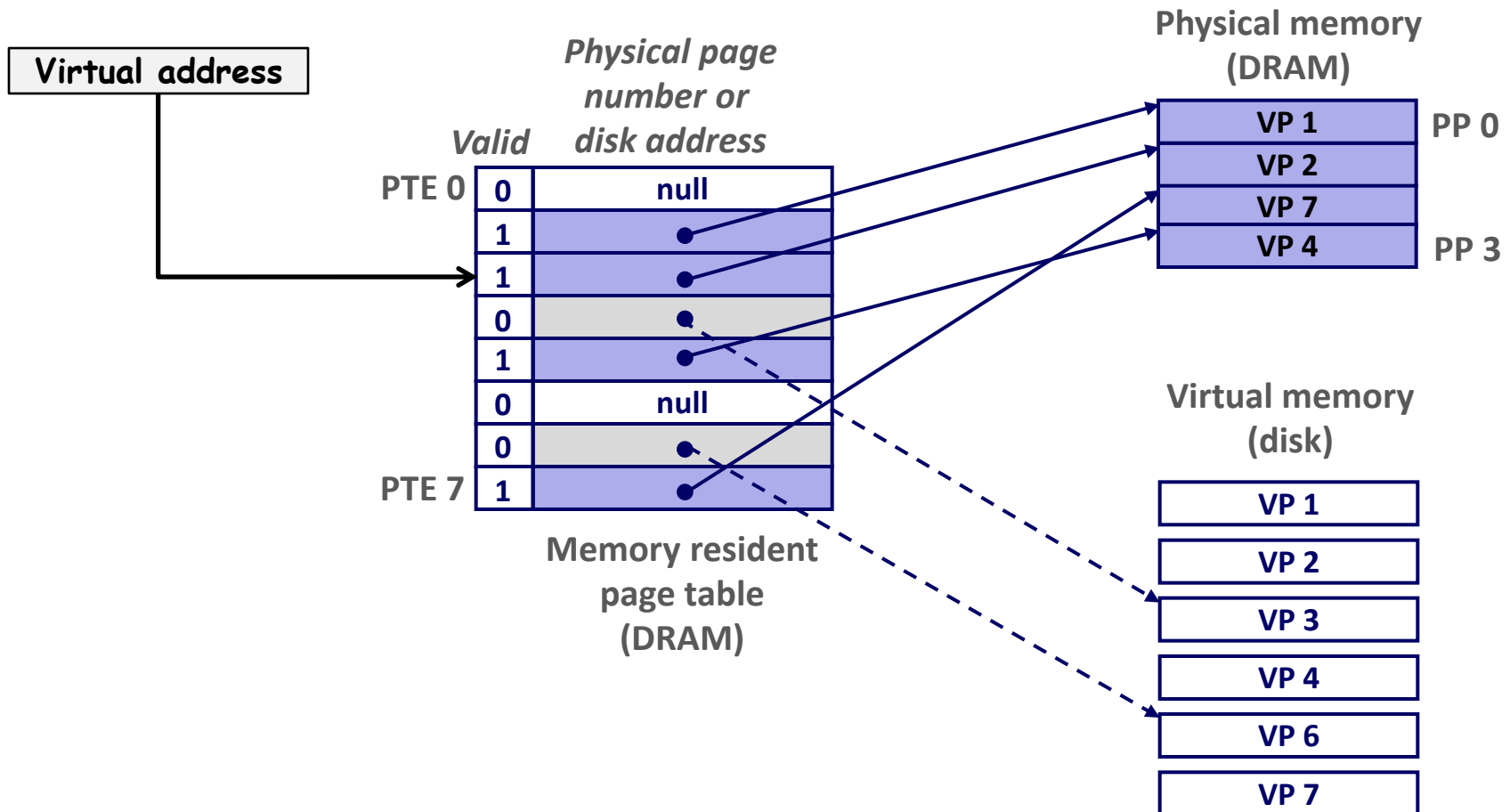
Enabling Data Structure: Page Table

- A *page table* is an array of page table entries (PTEs) that maps virtual pages to physical pages.
 - Per-process kernel data structure in DRAM



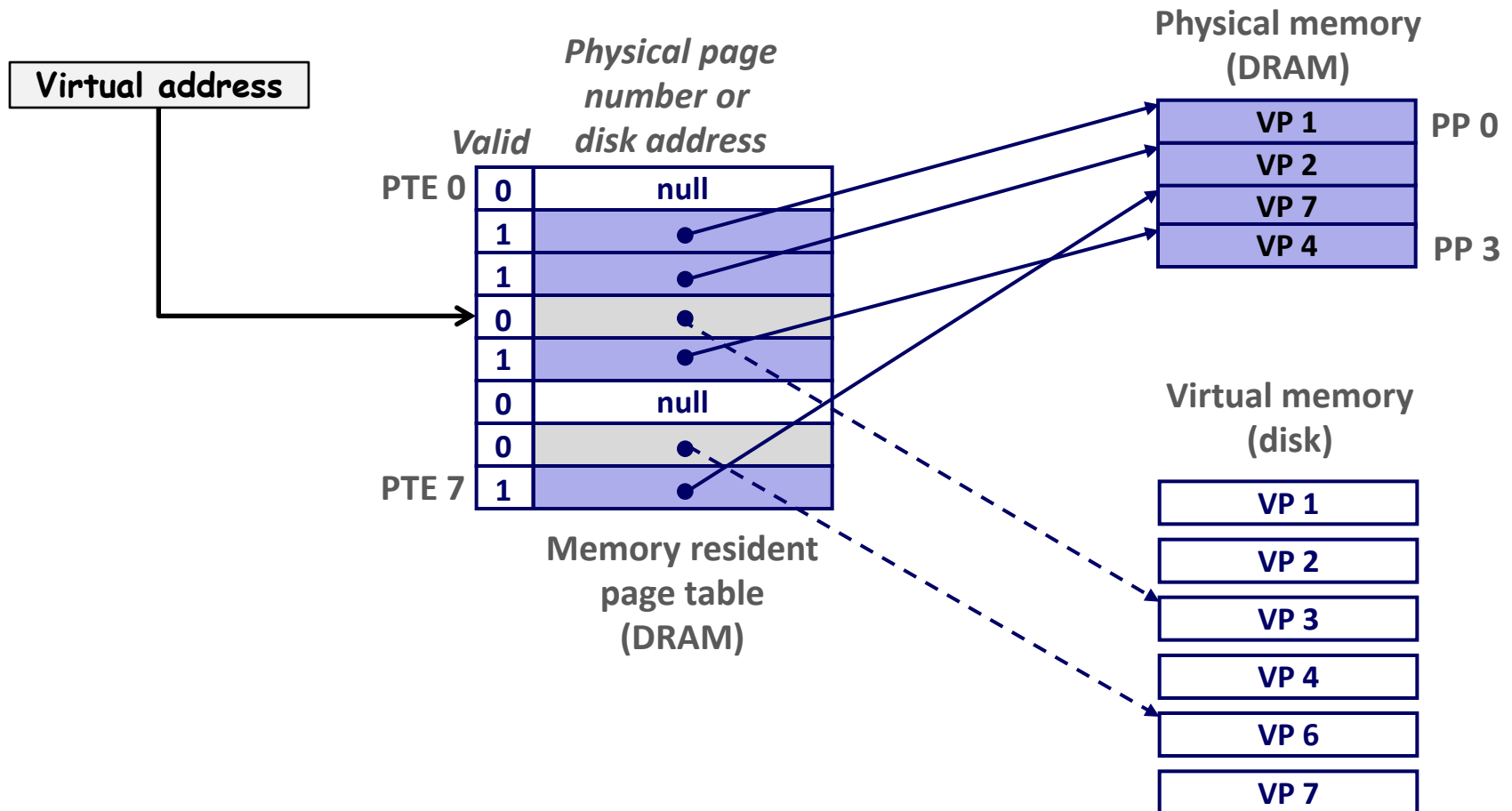
Page Hit

- *Page hit*: reference to VM word that is in physical memory (DRAM cache hit)



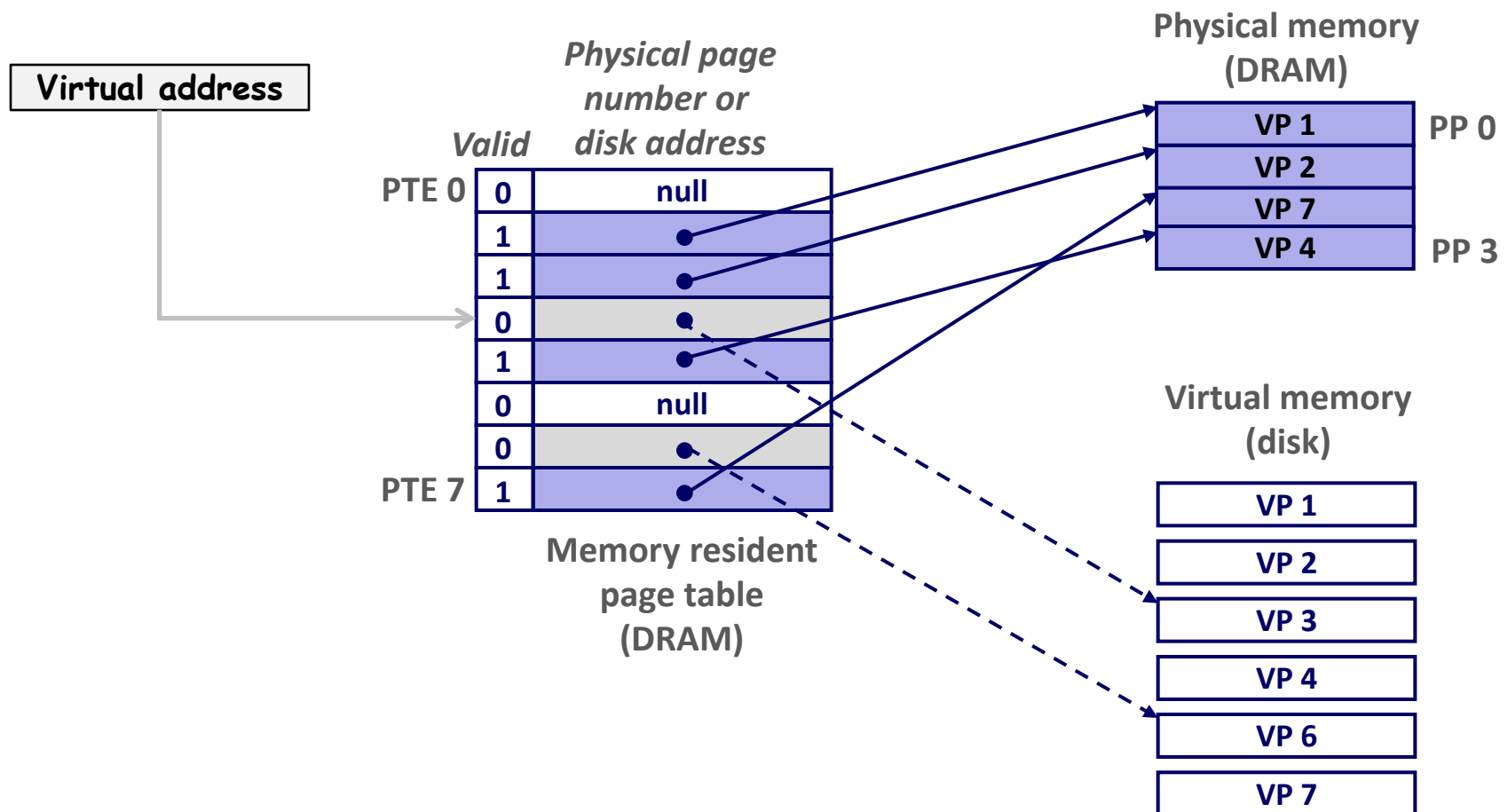
Page Fault

- *Page miss*: reference to VM word that is not in physical memory (DRAM cache miss)



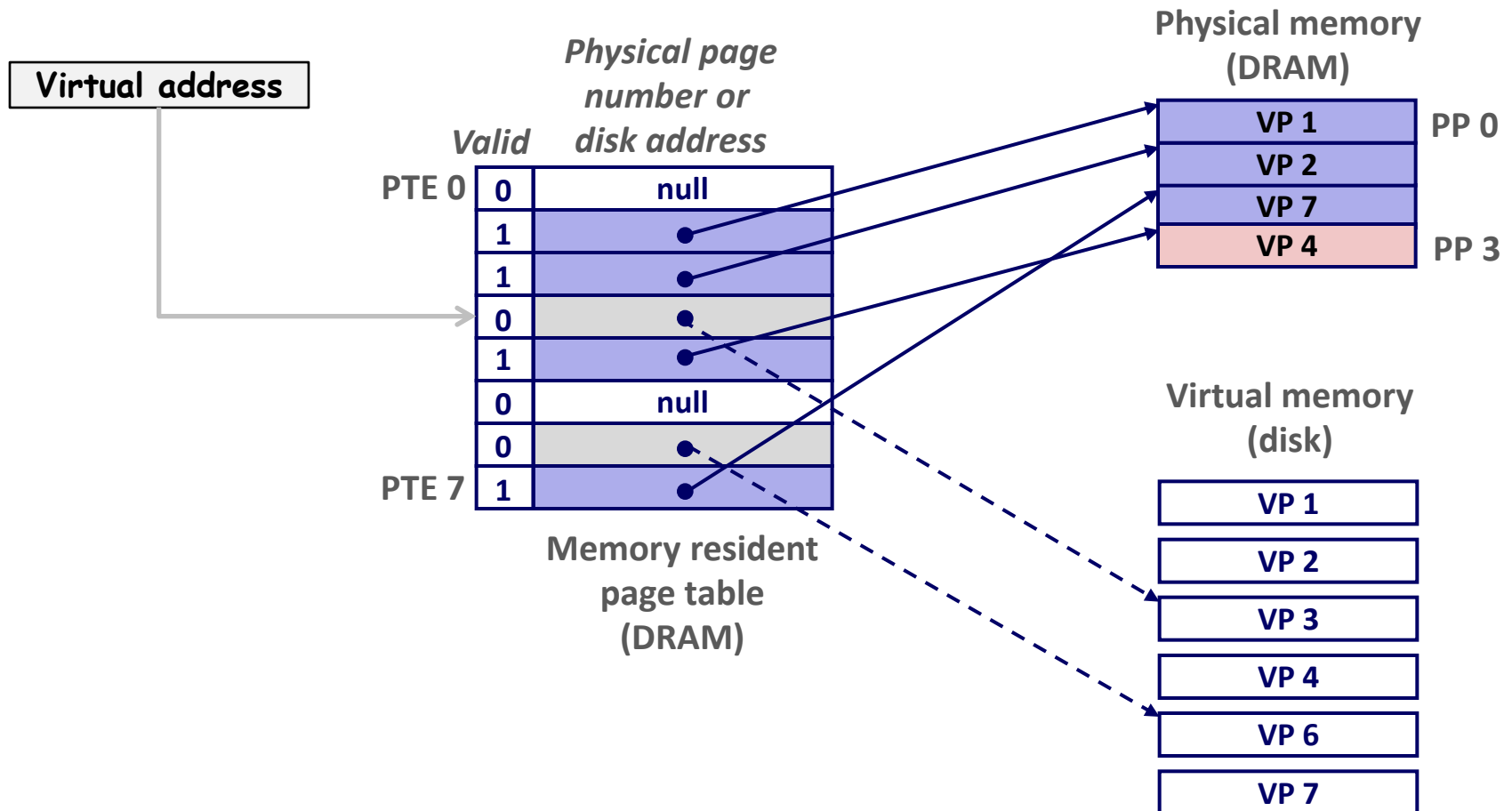
Handling Page Fault

- Page miss causes page fault (an exception)



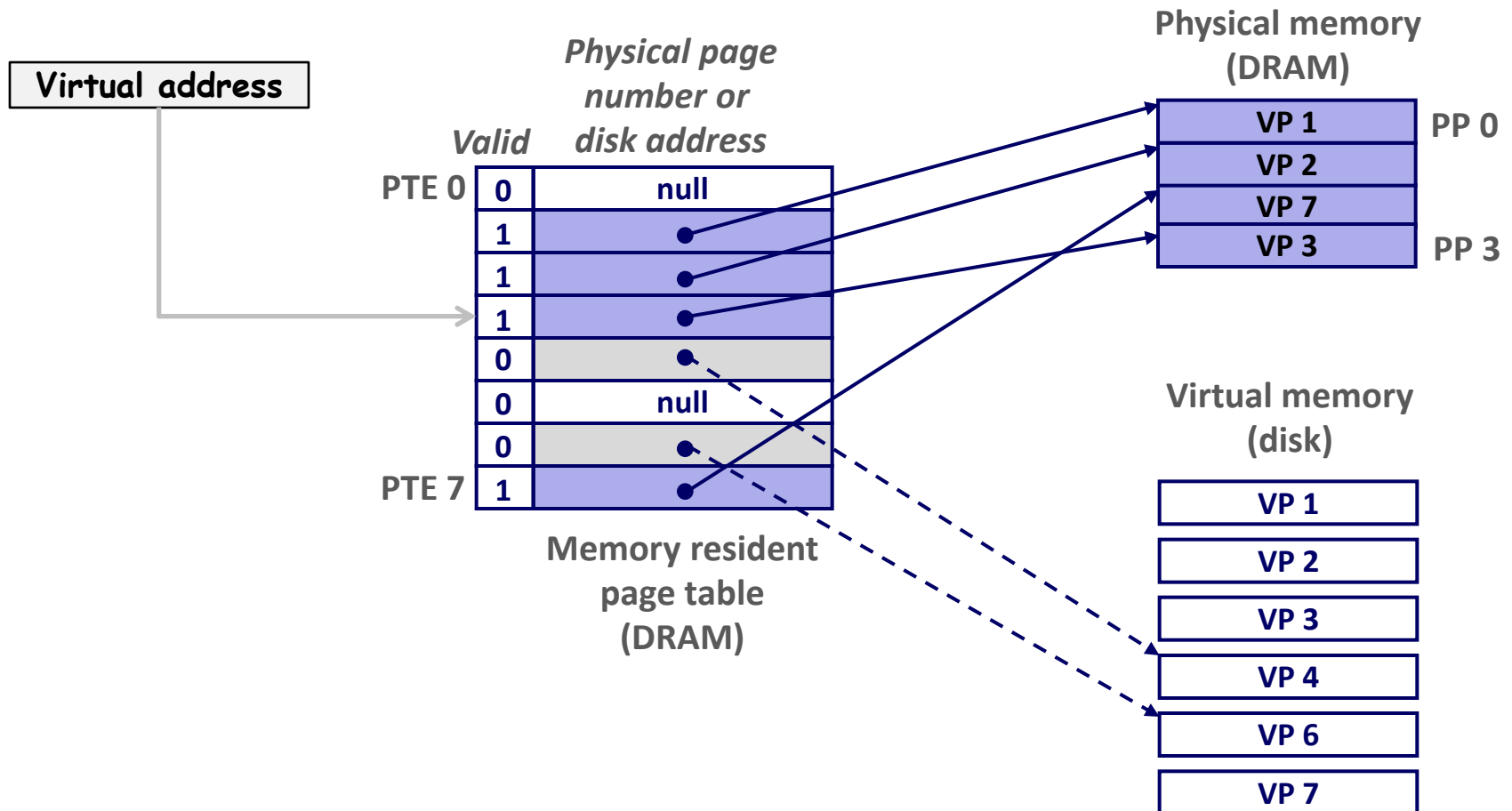
Handling Page Fault

- Page miss causes page fault (an exception)
- Page fault handler selects a victim to be evicted (here VP 4)



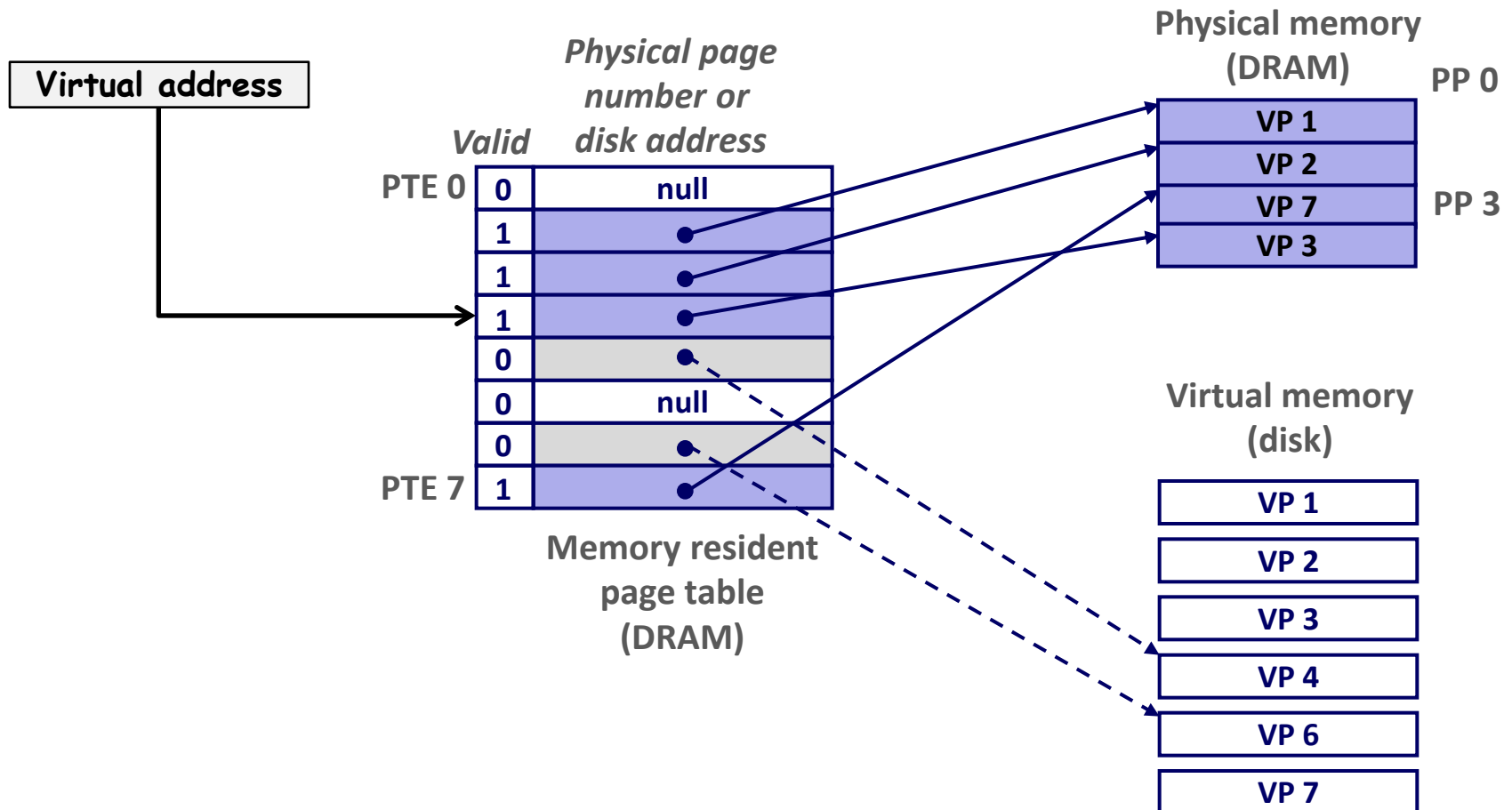
Handling Page Fault

- Page miss causes page fault (an exception)
- Page fault handler selects a victim to be evicted (here VP 4)



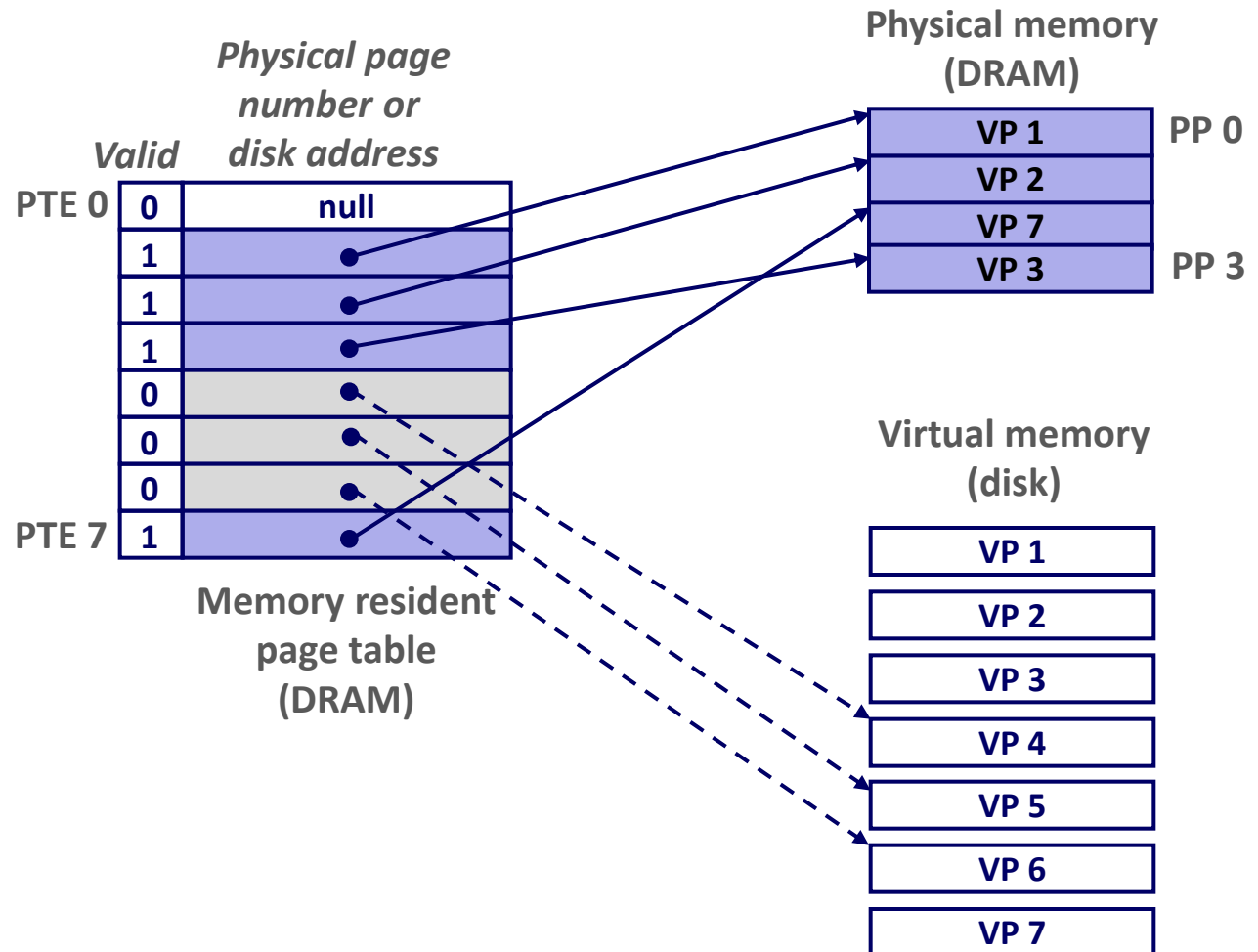
Handling Page Fault

- Page miss causes page fault (an exception)
- Page fault handler selects a victim to be evicted (here VP 4)
- Offending instruction is restarted: page hit!



Allocating Pages

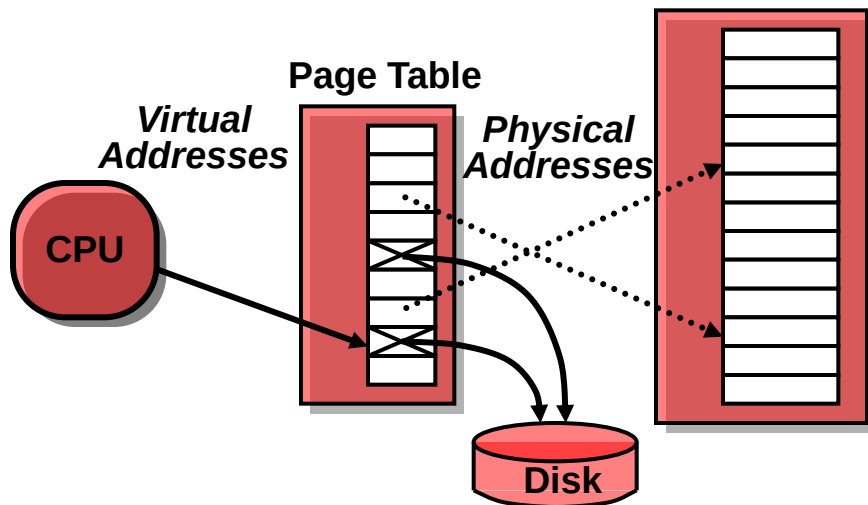
- Allocating a new page (VP 5) of virtual memory.



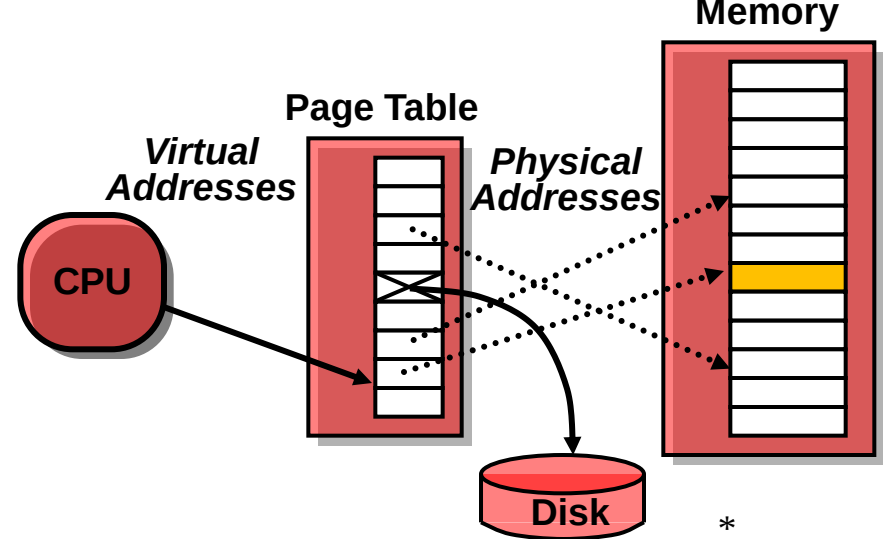
Page Faults

- Swapped out or paged out
- Demand paging(按需页面调度--lazy)

Before fault

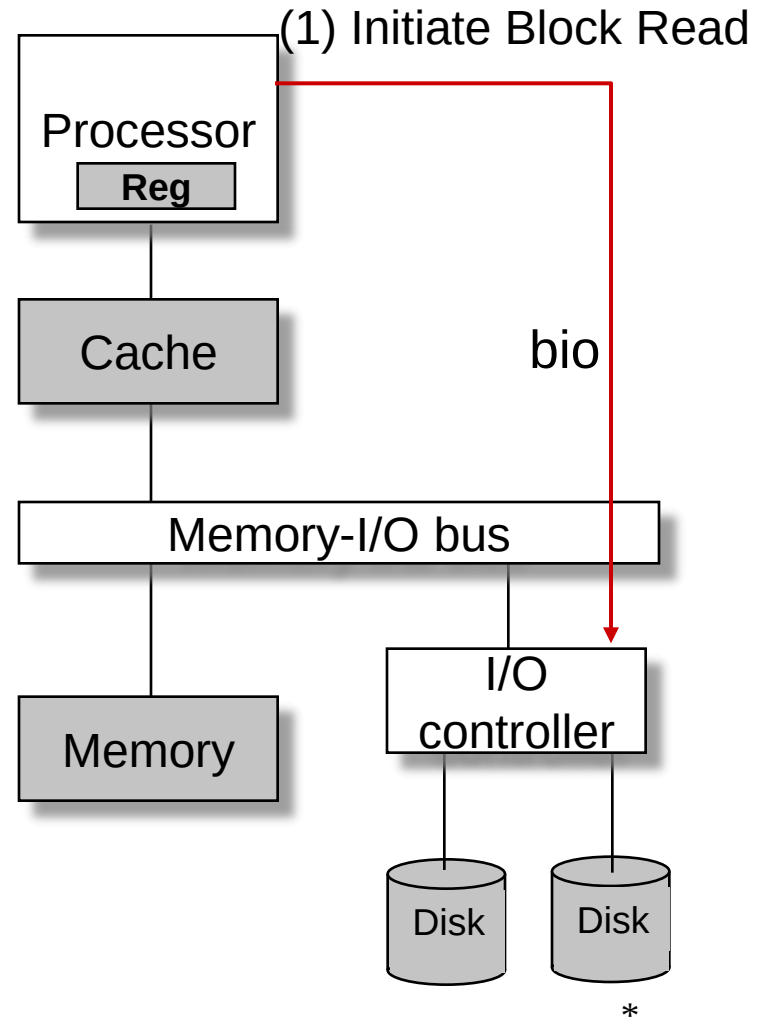


After fault



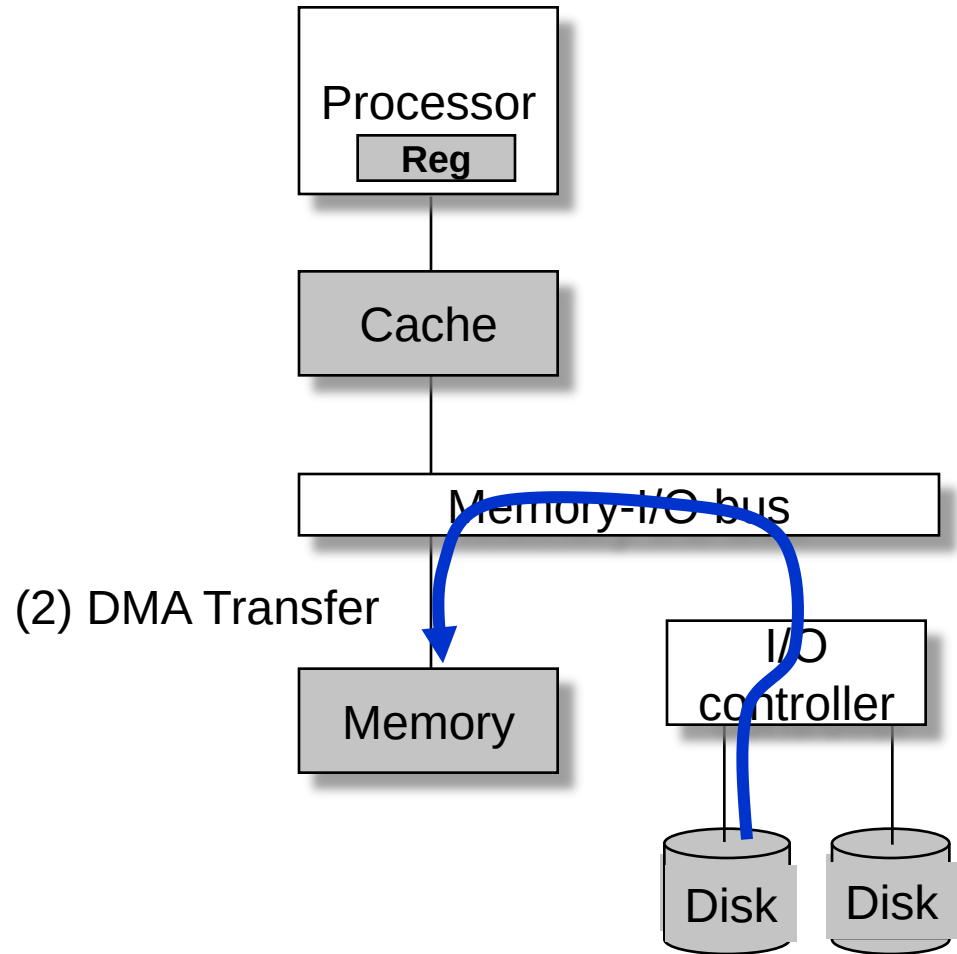
Servicing a Page Fault

- CPU signals
- Memory controller
 - Read block of length P starting at **disk address X** and store starting at **memory address Y**



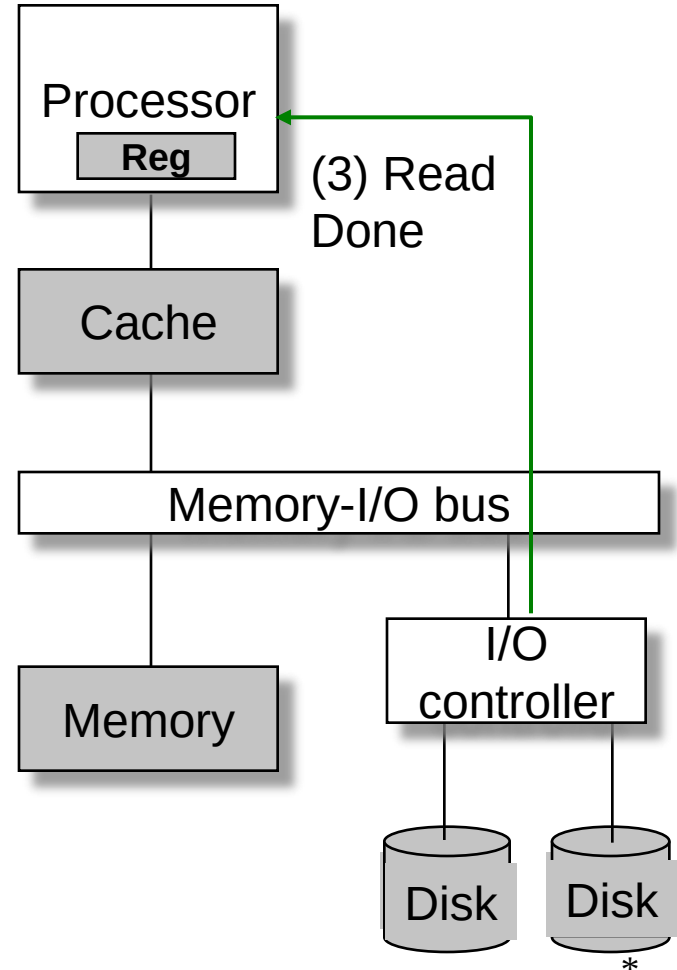
Servicing a Page Fault

- Read Occurs
 - Direct Memory Access (DMA)
 - Under control of I/O controller



Servicing a Page Fault

- I / O Controller Signals Completion
 - **Interrupt** processor
 - OS resumes suspended process



Locality to the Rescue Again!

- Virtual memory works because of **locality**
- At any point in time, programs tend to access a set of **active** virtual pages called the **working set**
 - Programs with better **temporal** locality will have smaller working sets

Locality to the Rescue Again!

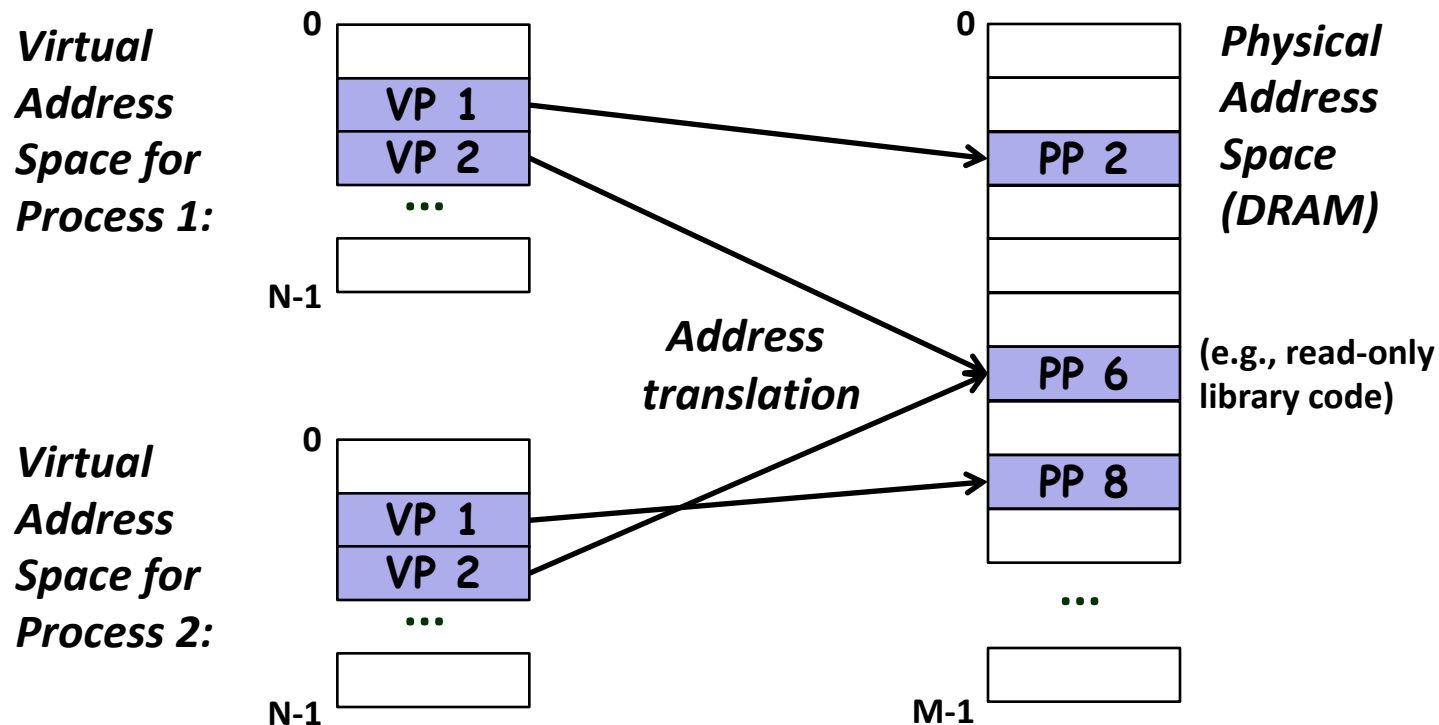
- If (working set size $<$ main memory size)
 - Good performance for one process after compulsory misses
- If (SUM(working set sizes) $>$ main memory size)
 - **Thrashing(颠簸)**: Performance meltdown where pages are swapped (copied) in and out continuously

Why Virtual Memory (VM)?

- Uses main memory efficiently
 - Use DRAM as a cache for the parts of a virtual address space
- Simplifies memory management
 - Each process gets the same uniform linear address space
- Isolates address spaces
 - One process can't interfere with another's memory
 - User program cannot access privileged kernel information

VM as a Tool for Memory Management

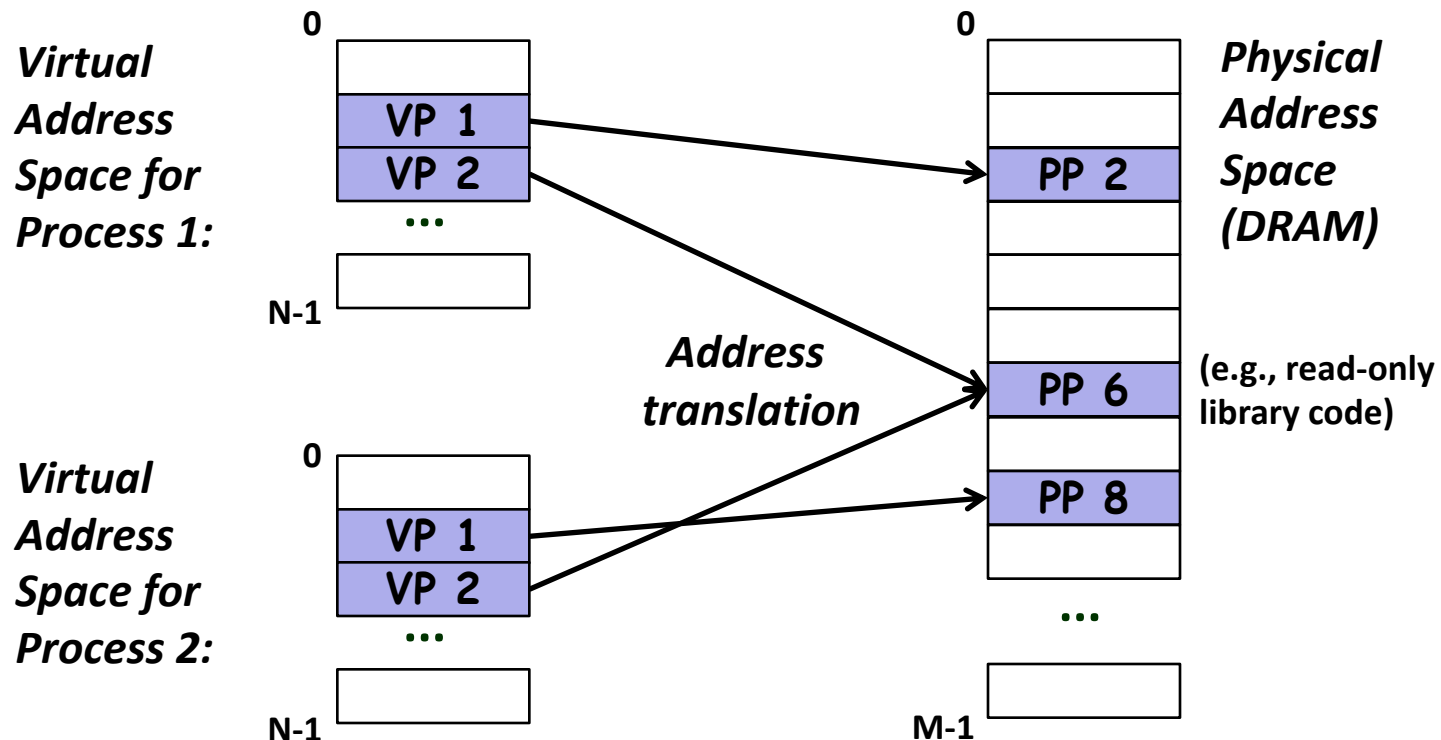
- Key idea: each process has its **own** virtual address space
 - 分层思想：链接、加载、共享、内存分配等很多操作和进程空间相关，于是每个进程的内存空间一样；逻辑页->物理页单独管理，与以上这些操作无关
 - 否则：用户进程在做内存访问时，都需要考虑是否在内存中



VM as a Tool for Memory Management

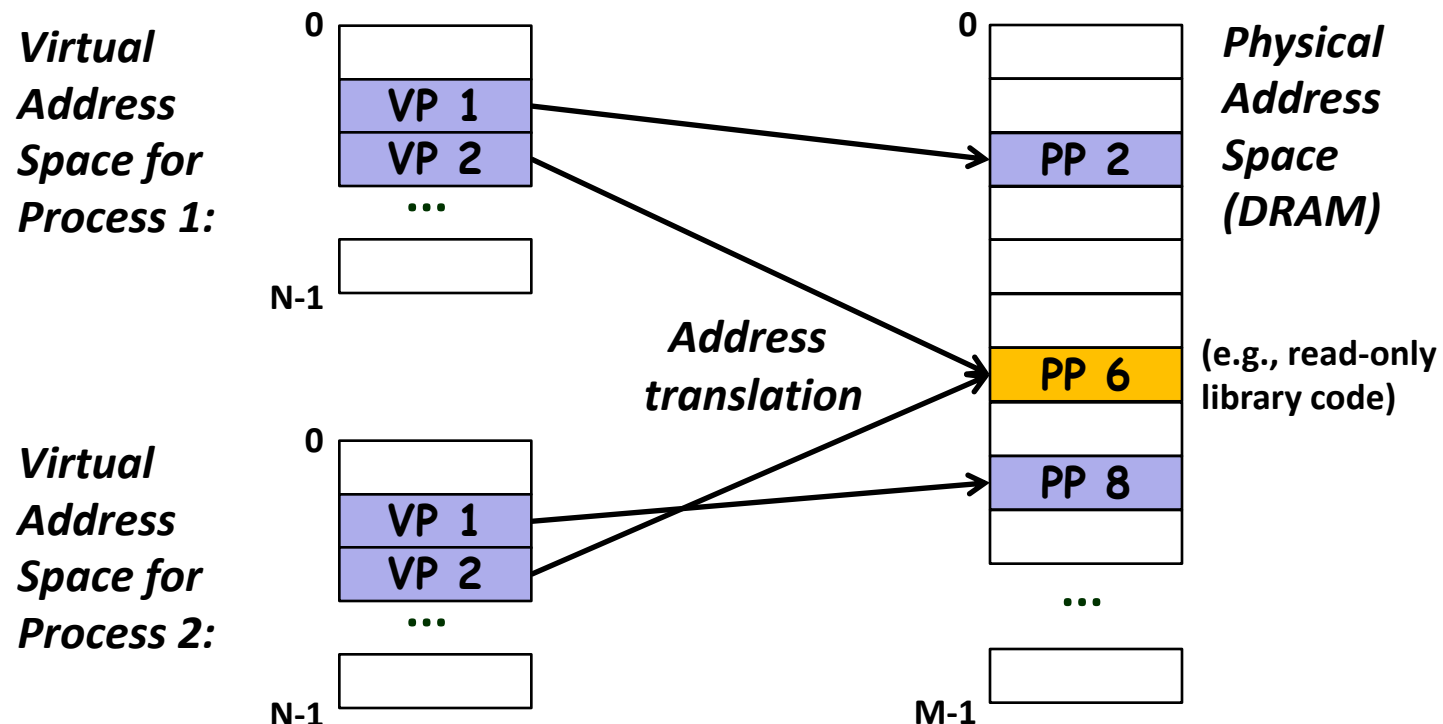
- Memory **allocation**

- Each virtual page can be mapped to any physical page
- A virtual page can be stored in different physical pages at different times
- 由于有页表，分配时看起来连续的内存，物理内存页未必连续



VM as a Tool for Memory Management

- **Sharing** code and data among processes
 - Map virtual pages to the same physical page

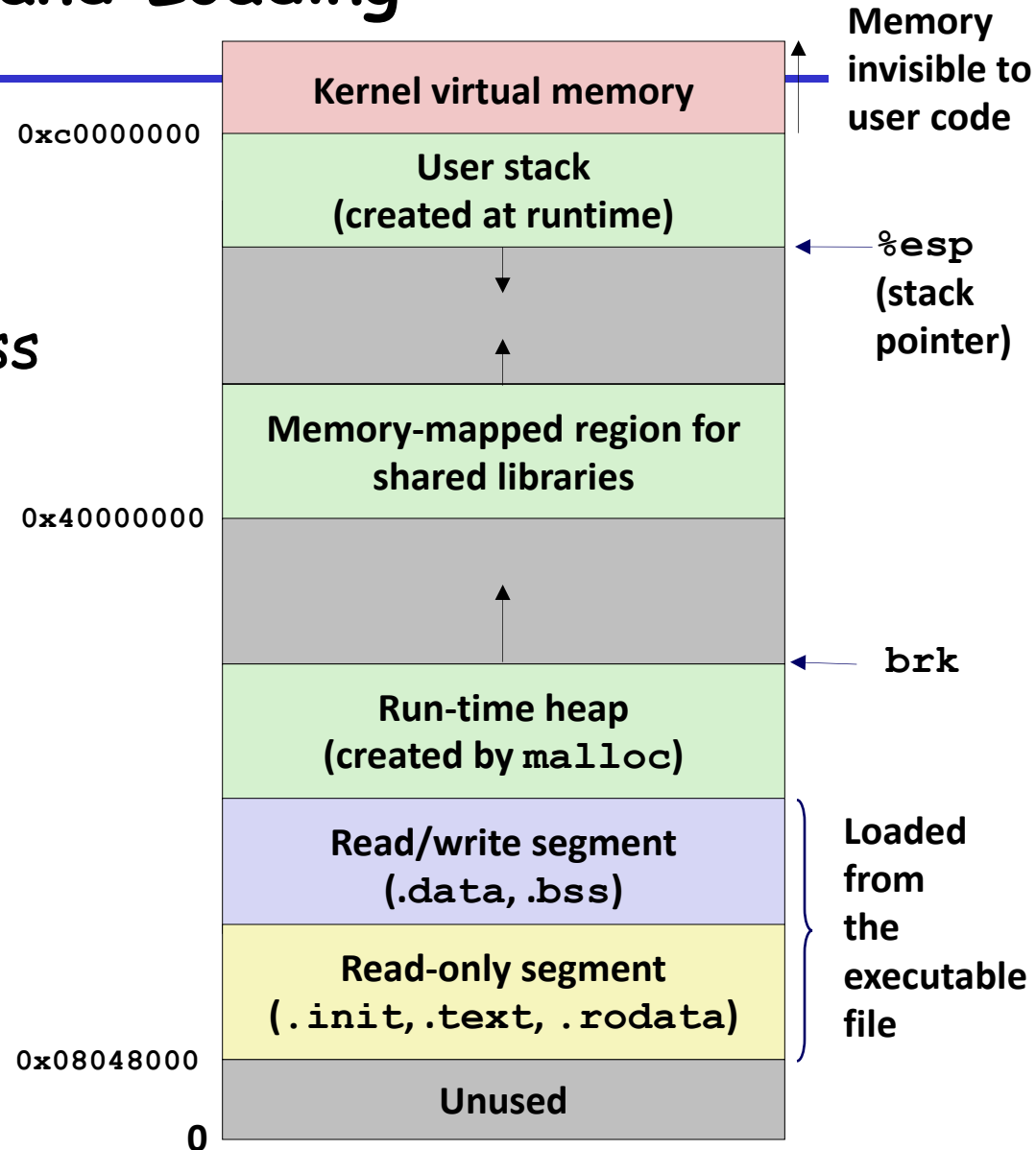


Simplifying Linking and Loading

- Linking

- Each program has **similar** virtual address space
- Code, stack, and shared libraries always start at the **same** address

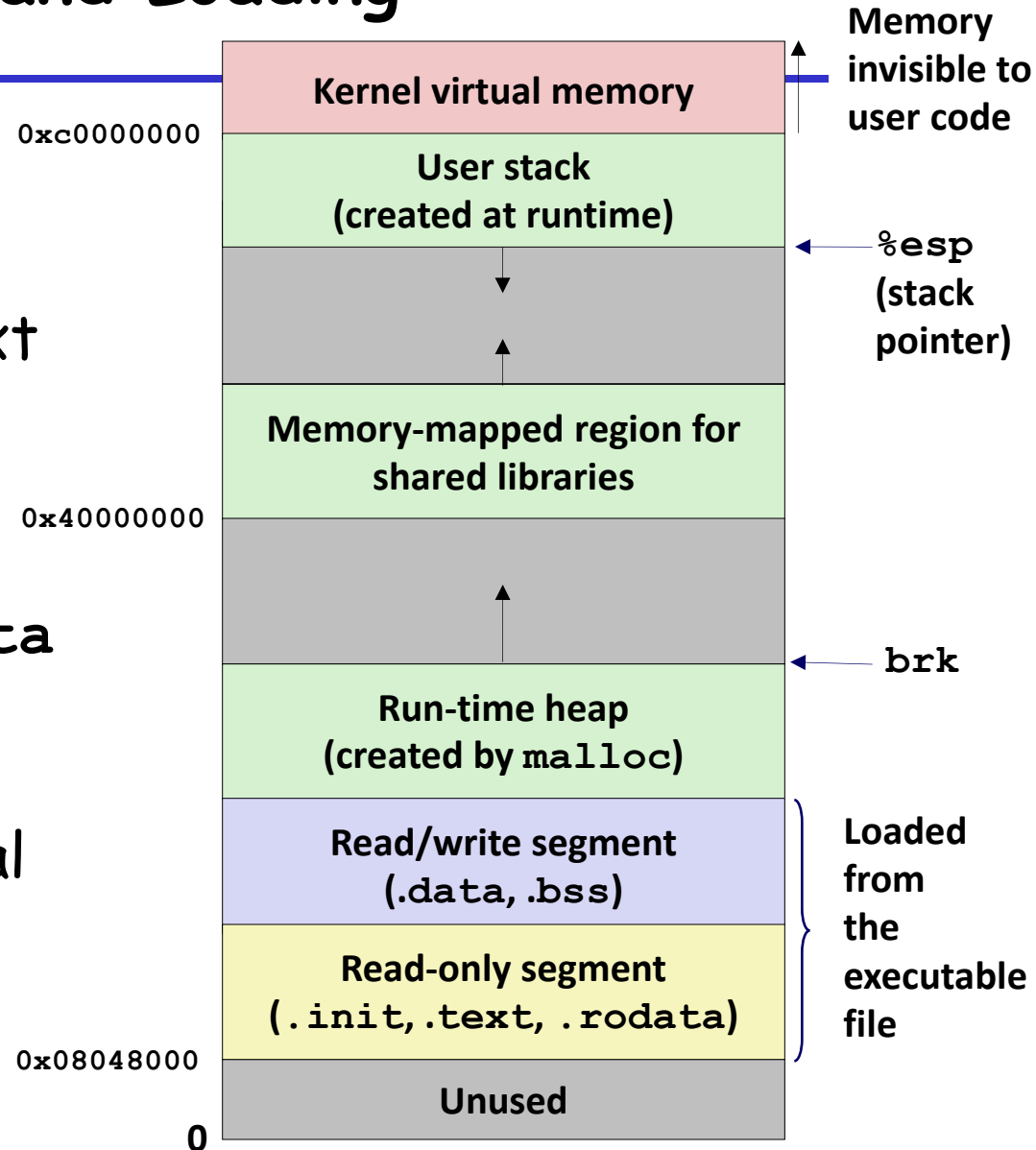
否则用户程序还要关心
自己程序所在的物理地址



Simplifying Linking and Loading

- Loading

- `execve()` allocates virtual pages for `.text` and `.data` sections = creates PTEs marked as **invalid**
- The `.text` and `.data` sections are copied, page by page, **on demand** by the virtual memory system

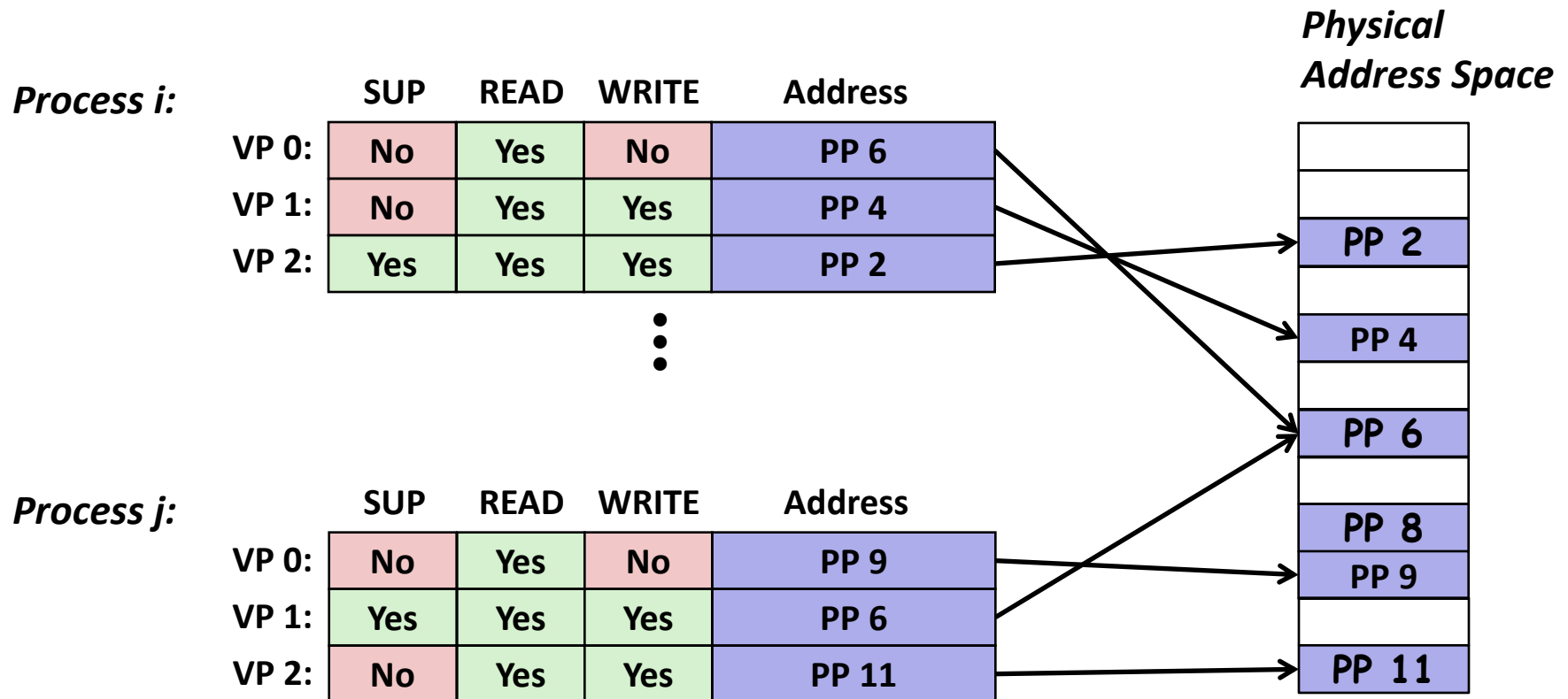


Why Virtual Memory (VM)?

- Uses main memory efficiently
 - Use DRAM as a cache for the parts of a virtual address space
- Simplifies memory management
 - Each process gets the same uniform linear address space
- Isolates address spaces
 - One process can't interfere with another's memory
 - User program cannot access privileged kernel information

VM as a Tool for Memory Protection

- Extend PTEs with permission bits
 - The same physical page has **different permission** for different process



VM as a Tool for Memory Protection

- Page fault handler checks these before remapping
 - If violated, send process SIGSEGV (segmentation fault)

